

</ Sistemas Operacionais

Etapa 1 - Fundamentação

/>

} /> [

Tema 1: getnproc()

</ Temas

{01}

Introdução

{02}

Syscall Getnproc()

{03}

Algoritmo Round Robin

{04}

Métricas

{05}

Implementação no
xv6

</ Integrantes e temas

<u>Introducao - Ayato</u>	0 que é o xv6, objetivo do projeto, tema escolhido.
<u>Syscall getnproc[] - João Victor</u>	Syscall, processo e syscall getnproc()
<u>Round Robin - Tarcila</u>	Funcionamento do algoritmo, quantum, vantagens e limitações.
<u>Métricas - Maria Cecília</u>	Tempo de CPU, tempo de espera, número de execuções, quantidade de processos ativos.
<u>Como será implementado no xv6 - Tulio</u>	Onde será modificado o kernel, como a syscall será adicionada, coleda de métricas e comparação de resultados.

Ayato Botelho Lima 12321BCC043

João Victor Melo Silva 12321BCC040

Maria Cecilia Rodrigues Oliveira Rosa 12321BCC044

Tarcila Augusta de Andrade e Figueiredo 12321BCC025

Túlio Martins 12311BCC055

</>

Parte 1: Introdução

01

} /> [

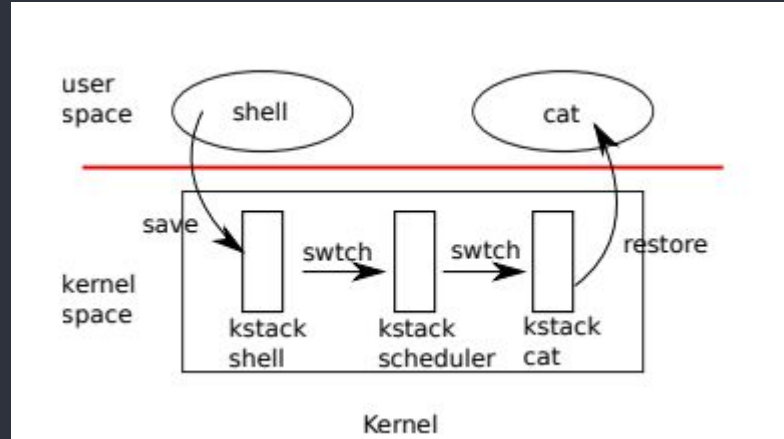
</ XV6

Foi criado pelo MIT em 2006 com o objetivo de ensinar o funcionamento interno de um sistema operacional moderno, ele é uma reimplementação mais nova e simplificada do Unix V6.

Tem como principais características o foco educacional, simplicidade unix, compatibilidade com os hardwares atuais e sem complexidade de produção.

Geralmente é executado dentro de ambientes emulados, onde ao inserir um código de um sistema operacional, é possível ver como o kernel é inicializado e testar comandos.

Figura 1: Trocando de um processo de usuário para outro, nesse exemplo o xv6 roda com uma CPU.



Fonte: xv6: a simple, Unix-like teaching operating system. Disponível em: <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>

</ Objetivo do projeto

Objetivo Geral: Implementar o syscall `getnproc()` no sistema operacional xv6 para contabilizar processos ativos e utilizá-la na análise experimental do comportamento do escalonador Round Robin.

Objetivos Especificos:

- Estudar a arquitetura do xv6;
- Desenvolver a syscall `getnproc()` e integrá-la no kernel do xv6;
- Coletar informações sobre os processos em execução;
- Avaliar o funcionamento do algoritmo Round Robin.

Tema > `getnproc()`

Semelhante à chamada de sistema `sysinfo()`, será criada com o objetivo de retornar a quantidade de processos ativos no sistema e auxiliar na análise do gerenciamento e escalonamento de processos. Poderá ser chamada por meio de programas de usuário.

</>

Parte 2: Syscall getnproc()

02

} /> [

</ XV6

O XV6 organiza seus processos por meio de uma struct chamada proc, onde é armazenado todas as informações essenciais de um processo. Quando um processo precisa de acesso a algo onde apenas o kernel pode acessar, é utilizado um mecanismo chamado syscall

```
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    int xstate;           // Exit status to be returned to parent's wait
    int pid;               // Process ID

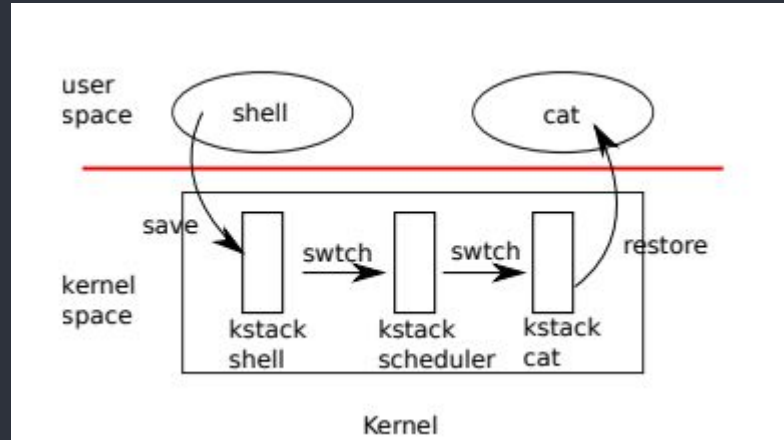
    // wait_lock must be held when using this:
    struct proc *parent; // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;        // Virtual address of kernel stack
    uint64 sz;            // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    struct trapframe *trapframe; // data page for trampoline.S
    struct context context; // swtch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    char name[16];          // Process name (debugging)
};
```


</ Xv6

Syscall é um jeito de processos em modo usuário acessar serviços que apenas o kernel tem acesso, por meio de uma solicitação a ele. O processo coloca o código da syscall desejada e executa a função `ecall`. O kernel recebe a syscall e executa ela, identificando a syscall na tabela pelo código dela e devolvendo o resultado para o processo.

Figura 1: Trocando de um processo de usuário para outro, nesse exemplo o xv6 roda com uma CPU.



Fonte: xv6: a simple, Unix-like teaching operating system. Disponível em: <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>

</ Xv6

Como xv6 usa uma tabela para guardar todos os processos do sistema. Uma forma de implementar a syscall seria utilizar essa tabela, fazendo uma função `getnproc()` que contaria com um `for` para percorrer toda a tabela e, o processo que não estivesse no estado `UNUSED`, seria contado e o valor final entregue como retorno na syscall. Uma função de syscall também deve ser implementada para ela.

</>

Parte 3: Algoritmo Round Robin (RR)

03

} /> [

</ Algoritmos de escalonamento

Necessidade

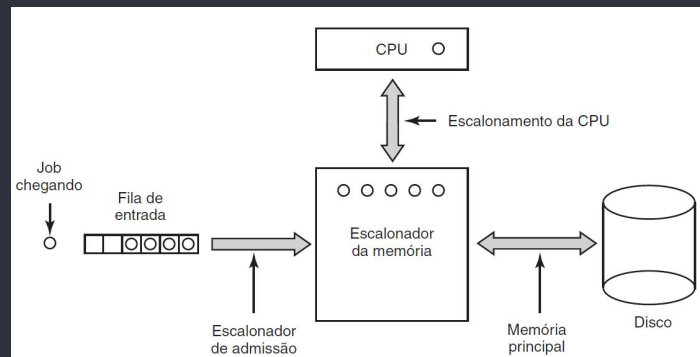
Multiprogramação significa muitos processos competindo por recursos.

Necessidade de escolher qual processo deve ser executado – escalonador faz essa escolha baseado num algoritmo de escalonamento

Objetivos

Otimizar o uso de recursos computacionais do sistema, maximizar a vazão (número de tarefas por hora que o sistema completa), minimizar o tempo ocioso, o tempo de execução de uma determinada tarefa, bem como o tempo entre a requisição e uma resposta (tempo de retorno).

Figura 2: Escalonamento



Fonte: LIMA, R. B. M. Escalonamento. Medium, 2021. Disponível em: <https://rbmelolima.medium.com/escalonamento-a72ec947579f>. Acesso em: 4 jun. 2026.

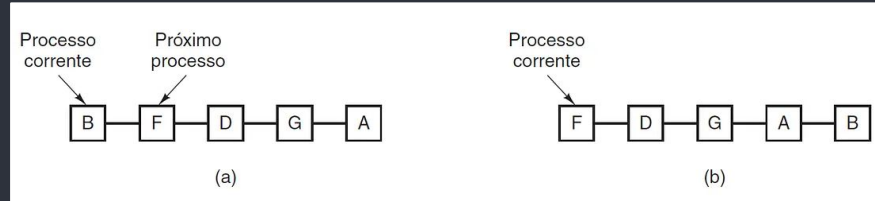
</ Alternância circular (Round Robin)

Funcionamento

Cada processo possui um intervalo de execução (quantum), que informa que, após esse tempo, o processo será preemptado (interrompido) e outro processo será executado.

O primeiro processo a chegar na fila de execução é atendido, e ele permanece em execução pela duração do quantum.

Figura 3: Lista de processos executáveis e lista de processos executáveis após o quantum do processo B ser consumido.



Fonte: LIMA, R. B. M. Escalonamento. Medium, 2021. Disponível em: <https://rbmelolima.medium.com/escalonamento-a72ec947579f>. Acesso em: 4 jun. 2026.



</ Alternância circular (Round Robin)

Exemplo



Tempos de cada programa

Programa	A	B	C	D
Ciclos	8	4	9	5

Cada processo (A, B, C e D) chega respectivamente nessa ordem

Como o algoritmo trabalha com first come first served, A começa a ser executado, e ele é executado até consumir seu quantum de 4 ciclos de CPU

Após os 4 ciclos ele é preemptado, B é executado, da mesma forma até terminar o quantum



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ Alternância circular (Round Robin)

Exemplo



Tempos de cada programa

Programa	A	B	C	D
Ciclos	8	4	9	5

Cada processo (A, B, C e D) chega respectivamente nessa ordem

Como o algoritmo trabalha com first come first served, A começa a ser executado, e ele é executado até consumir seu quantum de 4 ciclos de CPU

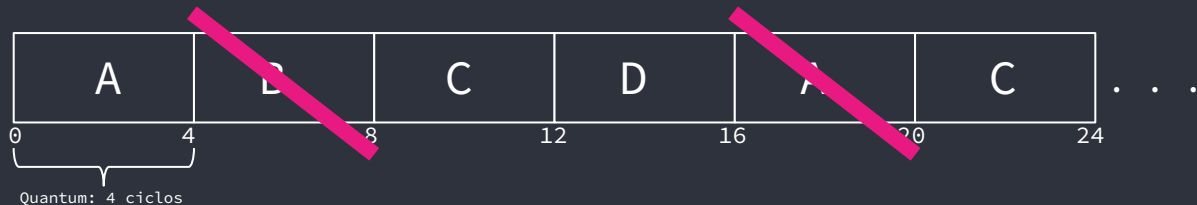
Após os 4 ciclos ele é preemptado, B é executado, da mesma forma até terminar o quantum



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1

</ Alternância circular (Round Robin)

Exemplo



Tempos de cada programa

Programa	A	B	C	D
Ciclos	8	4	9	5

Cada processo (A, B, C e D) chega respectivamente nessa ordem

Como o algoritmo trabalha com first come first served, A começa a ser executado, e ele é executado até consumir seu quantum de 4 ciclos de CPU

Após os 4 ciclos ele é preemptado, B é executado, da mesma forma até terminar o quantum



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1

</ Desvantagens

Overhead

Tempo de chaveamento de processos/contexto: tempo para chavear de um processo para outro pode ser grande (supondo 1ms) com um quantum curto (ex: 4ms, 20% do tempo da CPU será um overhead administrativo).

Grande número de solicitações

Com um quantum grande demais, processos no fim da lista podem demorar muito para executar.

</>

Parte 4: Métricas

04

} /> [

Métricas para avaliar o escalonador

Turnaround Time

- Tempo total:
chegada → término
- Fórmula:
 $\text{término} - \text{chegada}$
- Objetivo: MENOR

Waiting Time

- Tempo na fila de
prontos (sem
executar)
- Fórmula: $\text{turnaround} - \text{tempo de CPU}$
- Objetivo: MENOR

THROUGHPUT

- Processos
completados por
unidade de tempo
- Fórmula: $\frac{\text{processos}}{\text{tempo total}}$
- Objetivo: MAIOR

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Coletando Métricas no xv6

1. Struct proc [kernel/proc.h]

- int ctime;
// tempo de criação
- int rtime;
// tempo de CPU
usado
- int etime;
// tempo de término

2. scheduler()

- A cada tick:
rtime++ do processo
rodando
- wtime++ para
processos RUNNABLE

3. Syscall getpinfo()

- Retorna
métricas para
usuário
- Programa de
teste coleta os
dados

COMPARAÇÃO: RR ORIGINAL vs. NOSSO ALGORITMO

Experimento:

1. CARGA DE TESTE

- 2 processos CPU-bound [muito cálculo]
- 3 processos I/O-bound [muita espera]

2. PROCEDIMENTO

- Executar a MESMA carga duas vezes
- Rodada 1: Round Robin original do xv6
- Rodada 2: Nosso escalonador implementado

3. MÉTRICAS COMPARADAS

- Turnaround médio [menor]
- Waiting time médio [menor]
- Throughput [maior]

RESULTADOS ESPERADOS:

- Nosso algoritmo deve ter menor waiting time
- Melhor turnaround para processos interativos
- Throughput competitivo com o RR original

</>

Parte 5: Como será implementado no xv6

05

} /> [

1. Registro da Syscall



Definição

Adicionar `SYS_getnproc` em `syscall.h` e mapear o vetor em `syscall.c`.



Interface

Declarar `int getnproc(void)` em `user.h` para acesso pelo usuário.



User Space

Adicionar entrada em `usys.S` para gerar o *trap* via instrução `int`.

2. Lógica do getnproc()

Kernel Space (`proc.c`)

A função percorre a tabela global de processos (`ptable`) contando apenas os que não estão em estado `UNUSED`.

```
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){  
    if(p->state != UNUSED)  
        n++;  
}  
return n;
```


3. Coletando as Métricas



Execuções

ms

Tempo CPU

Modificação na struct proc

Adicionaremos campos na `struct proc` em `proc.h` para rastrear:

- `int exec_count;`
- `int wait_time;`
- `int cpu_time;`

Os dados serão atualizados a cada ciclo do escalonador Round Robin.

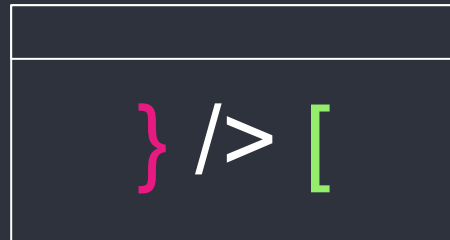
</ Referências:

LIMA, Rafael Bruno Melo. *Escalonamento*. Medium, 2021. Disponível em: <https://rbmelolima.medium.com/escalonamento-a72ec947579f>. Acesso em: 4 jun. 2026.

TANENBAUM, A. S. *Sistemas Operacionais: Projeto e implementação*. 3. ed. [S. l.: s. n.], 2008.

Xv6, a simple Unix-like teaching operating system. Disponível em: <https://pdos.csail.mit.edu/6.828/2012/xv6.html>

Wikipédia. XV6, 2026. Disponível em: https://en-wikipedia-org.translate.google/wiki/Xv6?x_tr_sl=en&x_tr_tl=pt&x_tr_hl=pt&x_tr_pto=tc



</ Referências:

MORRIS, Robert. Scheduling 2. Charlottesville: University of Virginia, Department of Computer Science, 10 fev. 2022. 57 slides. Notas de aula. Disponível em: <http://www.cs.virginia.edu/~cr4bd/4414/S2022/slides/20220210-slides.pdf>. Acesso em: 4 jun. 2026.

MORRIS, Robert. Scheduling 2. Charlottesville: University of Virginia, Department of Computer Science, 23 fev. 2021. 56 slides. Notas de aula. Disponível em: <http://www.cs.virginia.edu/~cr4bd/4414/S2021/slides/20210223-slides.pdf>. Acesso em: 4 jun. 2026.

KAASHOEK, Frans; MORRIS, Robert. xv6: a simple, Unix-like teaching operating system. Cambridge: MIT, 2024. 114 p. Disponível em: <https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf>. Acesso em: 4 jun. 2026.

